

16-Point FFT Radix 2^2 SDF

Mohamed Abdelhafez Elnewehy
Mohamed Mustafa Shiple

- ▶ About Radix 2²
- ▶ System Modeling and FxPt Design
- ▶ Architecture and RTL
- ▶ Verification (SV and CoCotb)
- ▶ FPGA Implementation
- ▶ ASIC Implementation
- ▶ Future Work
- ▶ References

About Radix 2²: The Best of Both Worlds

A hardware-oriented algorithm, derived by integrating a **twiddle-factor decomposition** technique into the divide-and-conquer approach.

RADIX-2

- ✓ Simple butterfly
- ✗ More stages → more multipliers

RADIX-4

- ✓ Fewer multipliers
- ✗ Complicated butterfly

RADIX-2²

- ✓ Simple radix-2 butterfly
- ✓ Radix-4 multiplicative complexity

Key idea: multiplications are arranged so that **only every other stage** performs a non-trivial multiplication — a structural advantage unique to pipeline/cascade FFT architectures.

MINIMAL HARDWARE REQUIREMENT (LENGTH-N DFT)

$$\log_4 N - 1 = 1$$

COMPLEX MULTIPLIERS

$$N - 1 = 15$$

COMPLEX DATA MEMORY

$$4\log_4 N = 8$$

COMPLEX ADDERS

About Radix 2²: Mathematical Derivation

N-POINT DFT DEFINITION

1

$$X(k) = \sum_{n=0}^{N-1} x(n) \cdot W_N^{nk}, \quad 0 \leq k < N$$

3-D LINEAR INDEX MAP

2

$$n = \langle (N/2)n_1 + (N/4)n_2 + n_3 \rangle_N \quad k = \langle k_1 + 2k_2 + 4k_3 \rangle_N$$

COMMON FACTOR ALGORITHM (CFA)

3

$$X(k_1 + 2k_2 + 4k_3) = \sum_{n_3} \{ B^{k_1}_{N/2}((N/4)n_2 + n_3) \cdot W_N^{\{((N/4)n_2 + n_3)k_1\}} \} \cdot W_N^{\{((N/4)n_2 + n_3)(2k_2 + 4k_3)\}}$$

RADIX-2 BUTTERFLY STRUCTURE B

4

$$B^{k_1}_{N/2}((N/4)n_2 + n_3) = x((N/4)n_2 + n_3) + (-1)^{k_1} x((N/4)n_2 + n_3 + N/2)$$

About Radix 2²: Mathematical Derivation

DECOMPOSED TWIDDLE FACTOR

5
$$W_n^{\{((N/4)n_2+n_3)(k_1+2k_2+4k_3)\}} = (-j)^{\{n_2(k_1+2k_2)\}} \cdot W_n^{\{n_3(k_1+2k_2)\}} \cdot W_{\{N/4\}}^{\{n_3k_3\}}$$

SET OF 4 DFTS OF LENGTH N/4

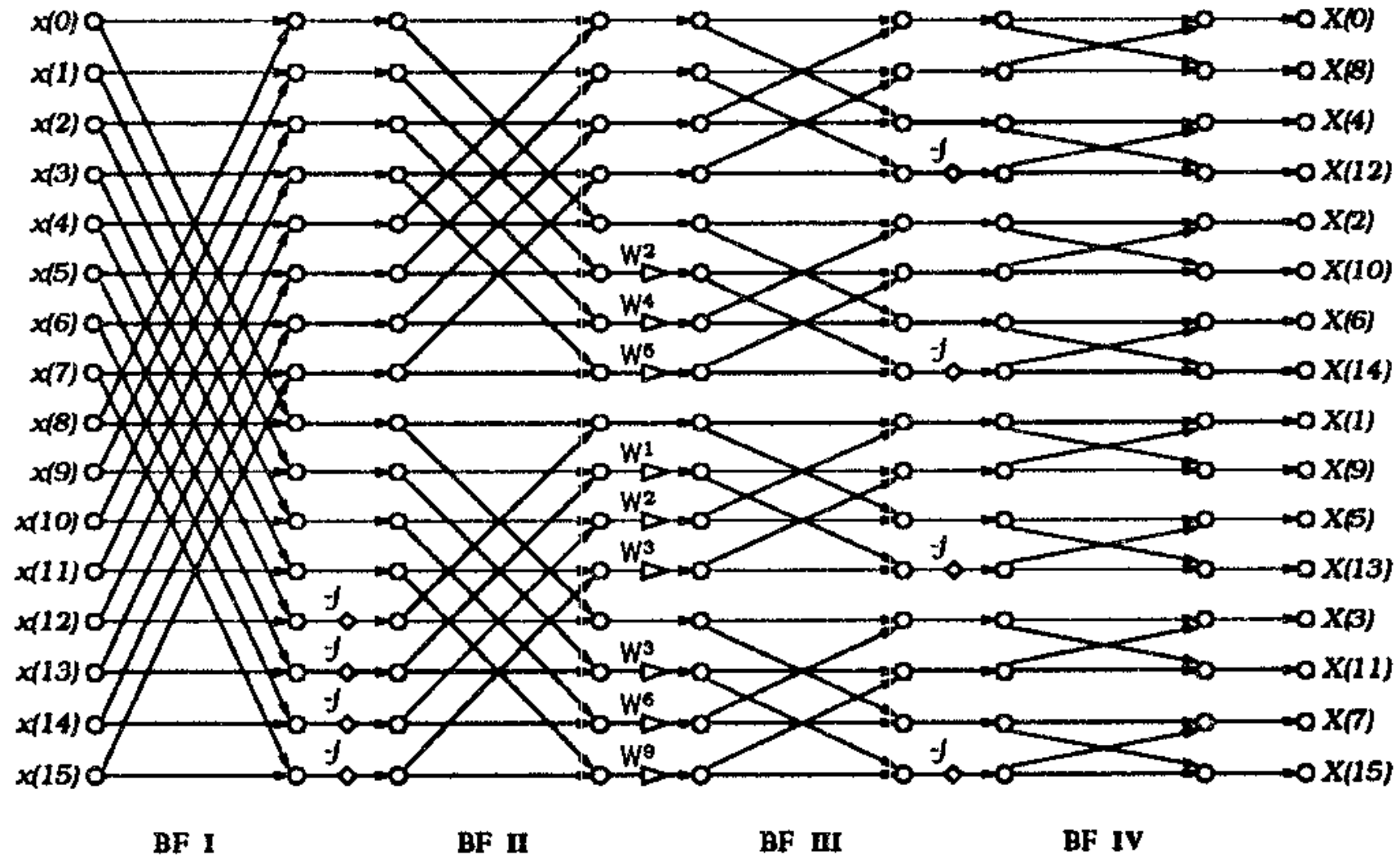
6
$$X(k_1+2k_2+4k_3) = \sum_{n_3} [H(k_1,k_2,n_3) \cdot W_n^{\{n_3(k_1+2k_2)\}}] \cdot W_{\{N/4\}}^{\{n_3k_3\}}$$

H(K₁,K₂,N₃) — THE RADIX-2² BUTTERFLY (CONCLUSION)

7
$$H = [\underbrace{x(n_3) + (-1)^{k_1}x(n_3+N/2)}_{\text{BF I}}] + (-j)^{\{(k_1+2k_2)\}} [\underbrace{x(n_3+N/4) + (-1)^{k_1}x(n_3+3N/4)}_{\text{BF II}}]$$

↳ Only trivial (real/imaginary-swap) multiplications inside BF I / BF II — one full complex multiplier every two stages.

About Radix 2^2 : SFG – 16 Point



16-point

FFT Size

Radix-2² SDF

Pipelined Architecture

Natural-in / Bit-Reversed-out (NI-BRO)

1 Floating-Point Reference

- Double-precision MATLAB model of the Radix-2² SDF algorithm
- MATLAB built-in `fft()` used as the golden reference

2 Word-Length Exploration

- `fi()` fixed-point types swept across stages
- 4 candidate configs (Test_1 → Test_4) via `FFT_2_2_types.m`

3 Bit-True Fixed-Point Model

- Cycle-accurate MATLAB model (`FFT_2_2.m`)
- Matches RTL data types, shift regs & control logic

4 Monte-Carlo SQNR Validation

- 100 random seeds per configuration
- Compared vs. floating-point `fft()`; SQNR & error computed

Metric: SQNR (dB) = $10 \cdot \log_{10}(\text{Signal Power} / \text{Noise Power})$, averaged across seeds for each configuration.

Final Fixed-Point Configuration → RTL Implementation: the selected configuration (Test_4) was carried forward directly into the RTL design.

Word-Length Exploration & SQNR

Goal: find the minimum fixed-point precision that gives sufficient numerical accuracy while avoiding unnecessary hardware cost.

Configuration	Mean SQNR (dB)	Min SQNR (dB)	Max Error (%)
Double (ref.)	317.50	314.02	0.000000
Single	144.07	141.12	0.000151
Test_1	72.48	69.25	0.987264
Test_2	60.33	57.29	2.862045
Test_3	48.47	45.41	12.376218
Test_4 (Final)	60.27	55.60	2.933179

Why Test_4: Test_4 provides the best precision-to-area trade-off. Despite using a smaller input word length than Test_3, its fractional-bit allocation improves SQNR by ~12 dB.

Selected Configuration → RTL Implementation

FxPt Verification Results — Final Config

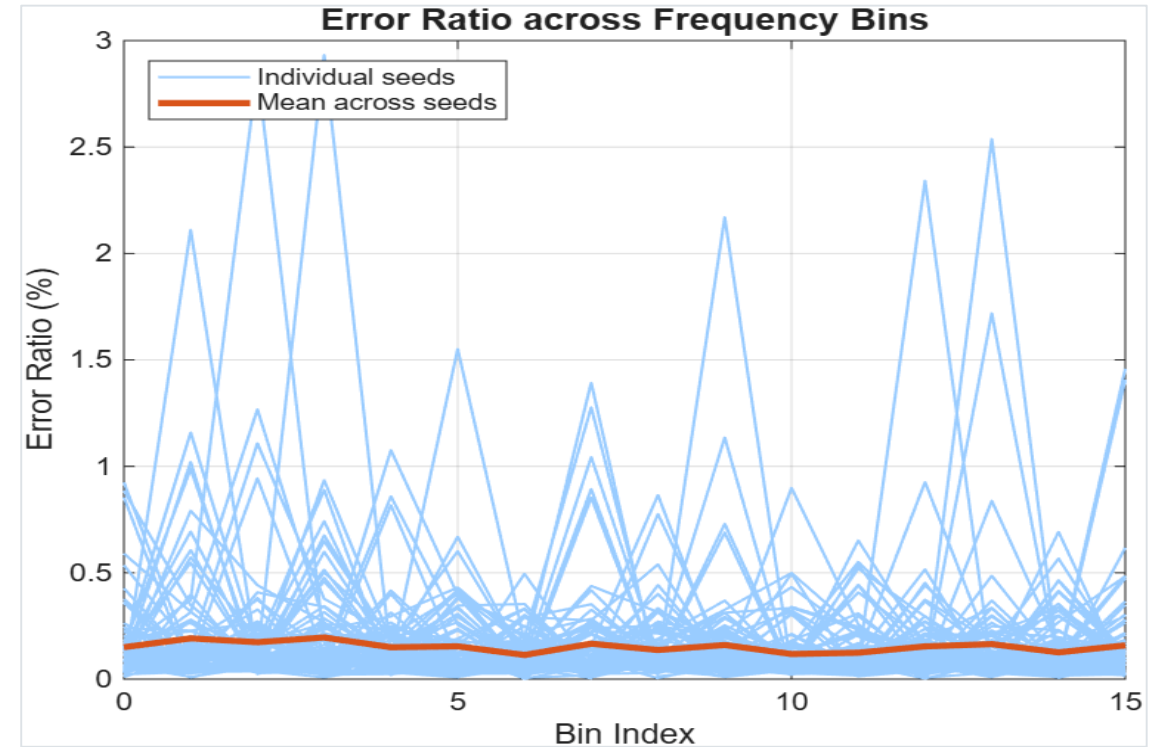
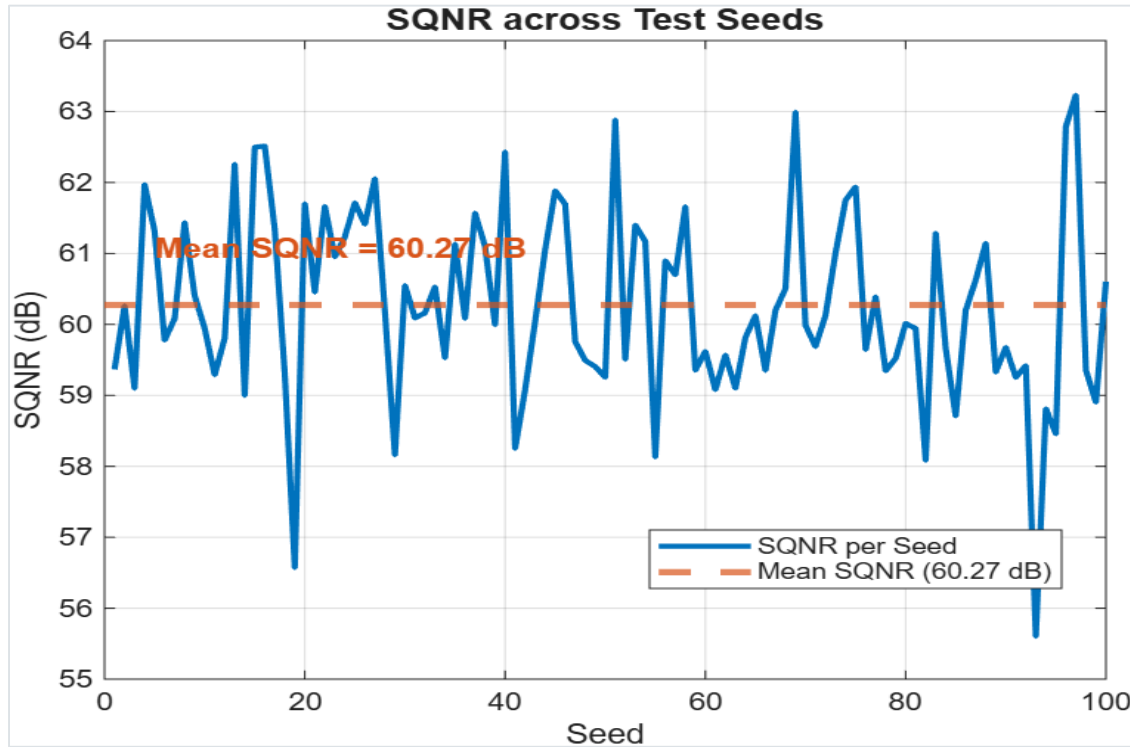
case 'Test_4'

```

T.x          = fi([],1,1+10,10);
T.b0         = double([]);
T.b1         = double([]);
T.b2         = double([]);
T.b3         = double([]);
T.sr8_out    = fi([],1,2+10,10);
T.sr4_out    = fi([],1,3+11,11);
T.sr2_out    = fi([],1,4+10,10);
T.sr1_out    = fi([],1,4+10,10);
T.sr8_in     = fi([],1,2+10,10);
T.sr4_in     = fi([],1,3+11,11);
T.sr2_in     = fi([],1,4+10,10);
T.sr1_in     = fi([],1,4+10,10);
T.bf1_out    = fi([],1,2+10,10);
T.bf2_out    = fi([],1,3+11,11);
T.bf3_out    = fi([],1,4+10,10);
T.bf4_out    = fi([],1,4+10,10);
T.twiddle_factor = fi([],1,1+11,11);
T.complex_mult_out = fi([],1,3+11,11);
T.y          = fi([],1,4+10,10);
    
```

Signal	WL	Total width	Int bits	Frac bits
x (input)	11	22	1	10
SR8 / sr8_in / sr8_out / bf1_out	12	24	2	10
SR4 / sr4_in / sr4_out / bf2_out	14	28	3	11
twiddle_factor	12	24	1	11
complex_mult_out	14	28	3	11
SR2 / sr2_in / sr2_out / bf3_out	14	28	4	10
SR1 / sr1_in / sr1_out / bf4_out	14	28	4	10
out_fft_raw / y (output)	14	28	4	10

FxPt Verification Results — Final Config



60.27 dB

Average SQNR (100 seeds)

55.60 dB

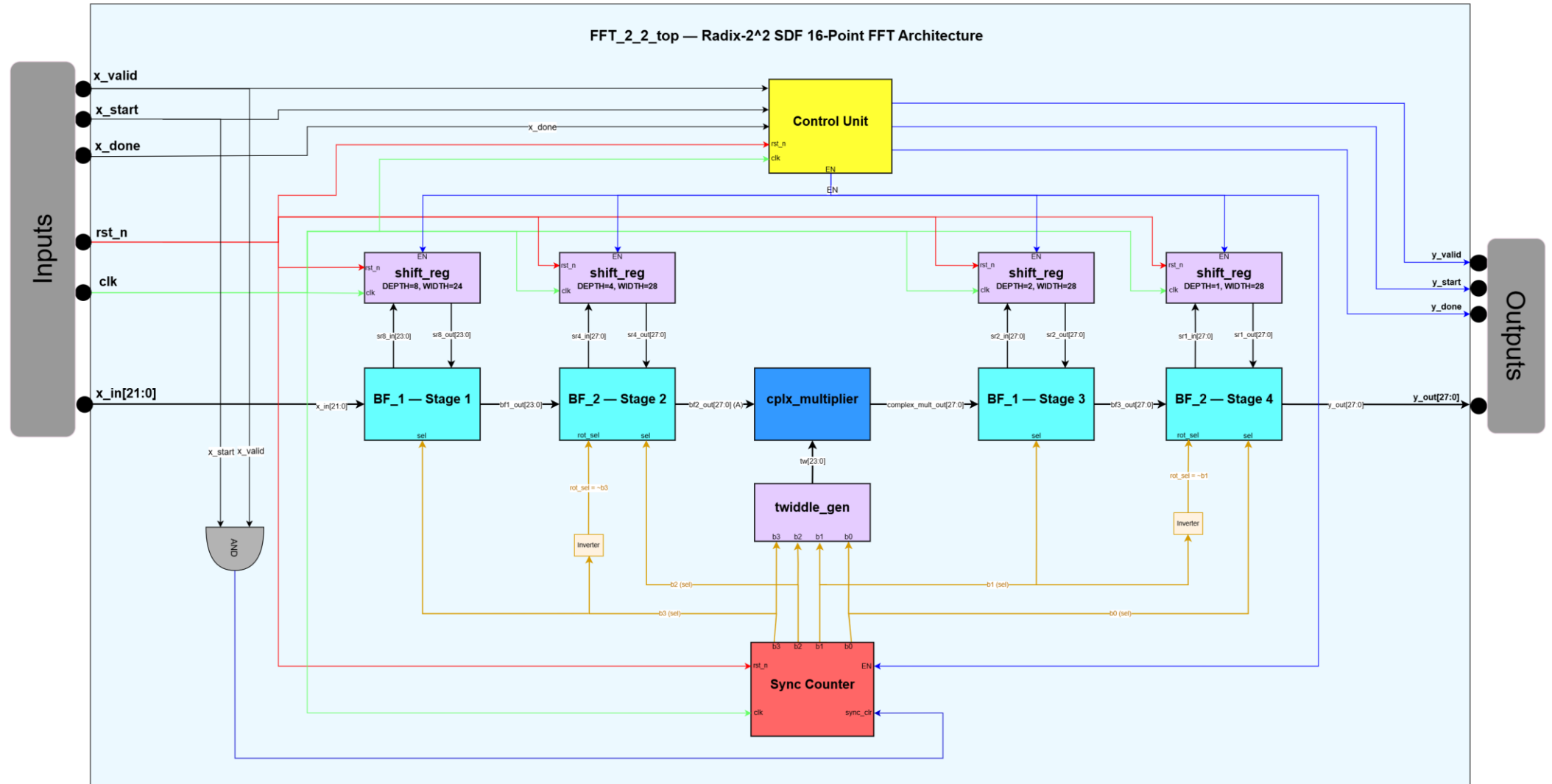
Min SQNR (worst seed)

2.93 %

Max Error Ratio (worst seed)

Fixed-point (Test_4) results validated against MATLAB floating-point fft() reference — high accuracy achieved, config carried forward to RTL.

Architecture and RTL



Algorithm 6.10:

Efficient Complex Multiplier

The complex twiddle factor multiplication $R + jI = (X + jY) \times (C + jS)$ can be simplified, because C and S are precomputed and stored in a table. It is therefore also possible to store the following three coefficients:

$$C, \quad C + S, \quad \text{and} \quad C - S. \quad (6.29)$$

With these three precomputed factors we first compute

$$E = X - Y, \quad \text{and then} \quad Z = C \times E = C \times (X - Y). \quad (6.30)$$

We can then compute the final product using

$$R = (C - S) \times Y + Z \quad (6.31)$$

$$I = (C + S) \times X - Z. \quad (6.32)$$

- ▶ **Complex Multiplier Pipelining:** Inserted a pipeline register stage inside the complex multiplier to isolate the multiplication phase from the addition phase, effectively halving the critical path delay.
- ▶ **Control Signal Synchronization (Delay Matching):** Added 1-cycle delay registers to the control signals (b1 and b0) feeding Stage 3 and Stage 4 to maintain perfect data-control alignment.
- ▶ **Global Latency Adjustment:** Updated the main control unit to increase the overall system latency by one clock cycle, seamlessly accommodating the new pipelined architecture.

16-point

FFT Size

Radix-2² SDF

Streaming Pipeline

Natural-in / Bit-Reversed-out (NI-BRO)

1 Delay / Shift-Register Chain

- 4 shift-register blocks (depths 8, 4, 2, 1 samples)
- Store samples between butterfly stages for in-place SDF data reuse

2 Butterfly Units

- 4 butterfly instances (BF_1 ×2, BF_2 ×2)
- Implements the BF I / BF II Radix-2² butterfly pair per stage

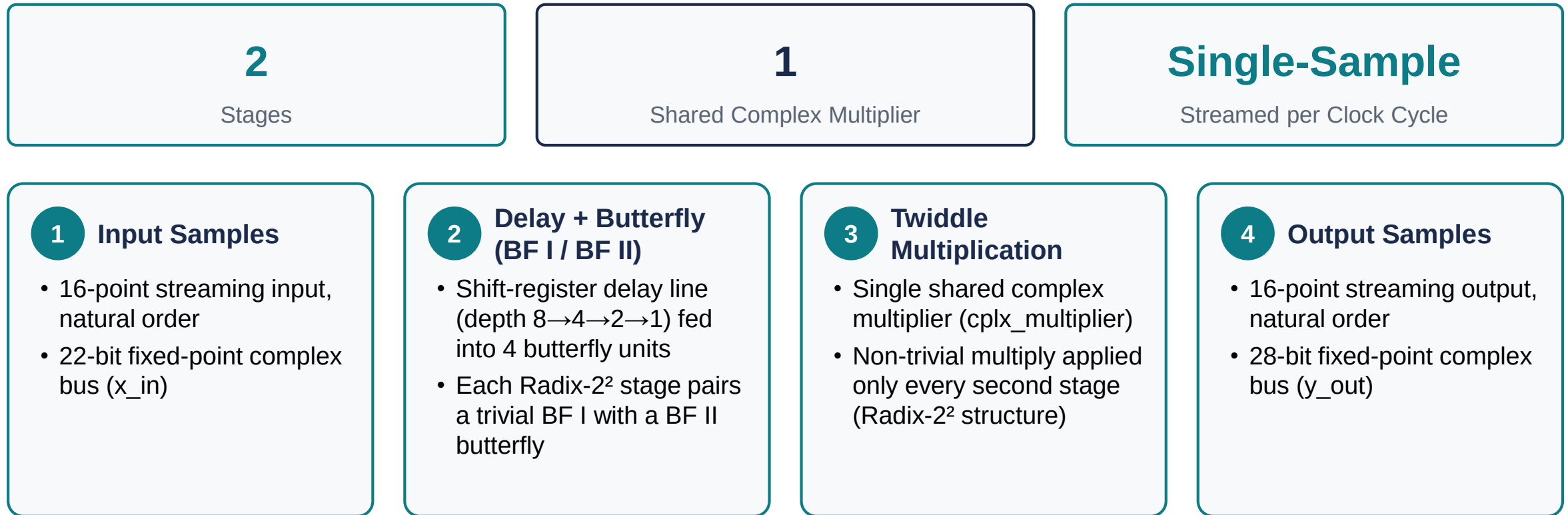
3 Complex Multiplier

- 1 shared complex multiplier (cplx_multiplier)
- Applies twiddle factors; reused across stages per Radix-2² decomposition

4 Control Logic

- control_unit + sync_counter blocks
- Sequences shift registers & butterflies; drives valid/done timing

Fixed-Point Model → **RTL**: the Test_4 fixed-point format defines the data types implemented in the butterfly and multiplier datapath below.



RTL Data Path → Verification: this datapath is exercised end-to-end by the SystemVerilog testbench (see Verification).

22-bit

Input Data Width (x_in)

28-bit

Output Data Width (y_out)

Active-Low

Reset Polarity (rst_n)

SDF

Streaming Interface Style

Input Interface

reset:

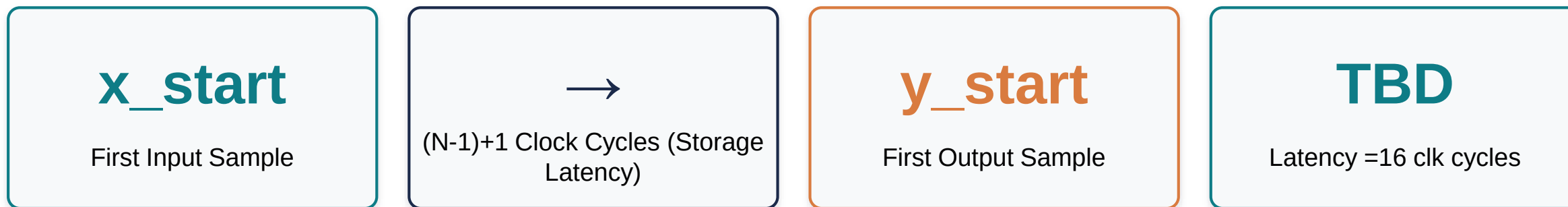
- rst_n — active-low reset

Data & Framing:

- x_in[21:0], x_start, x_valid, x_done

Output Interface

- y_out[27:0] — 28-bit fixed-point complex output sample bus
- y_start — pulses high at the first output sample of a frame
- y_valid / y_done — output sample valid / end-of-frame flags



Measurement Method

Definition:

- Cycles between x_start/x_valid on the first input sample and y_start/y_valid on the first output sample

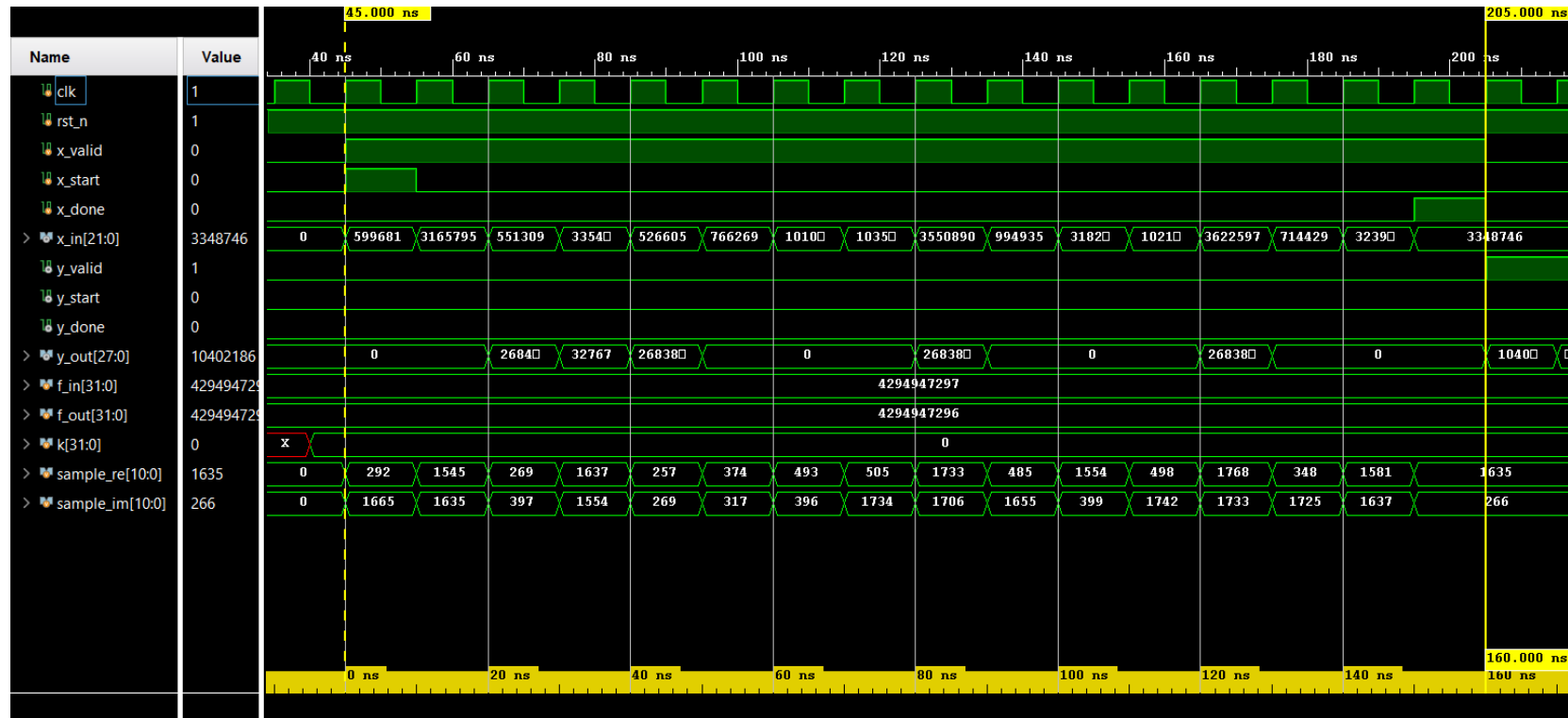
Source:

- Measured directly from the RTL simulation waveform / testbench log

Measured Results

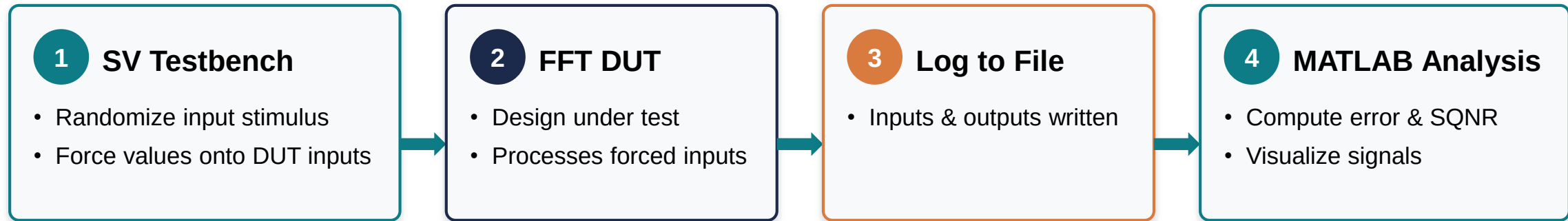
- Latency value is 16 clock cycles
- Total latency for the whole operation = 32 clock cycles

RTL Performance — Latency



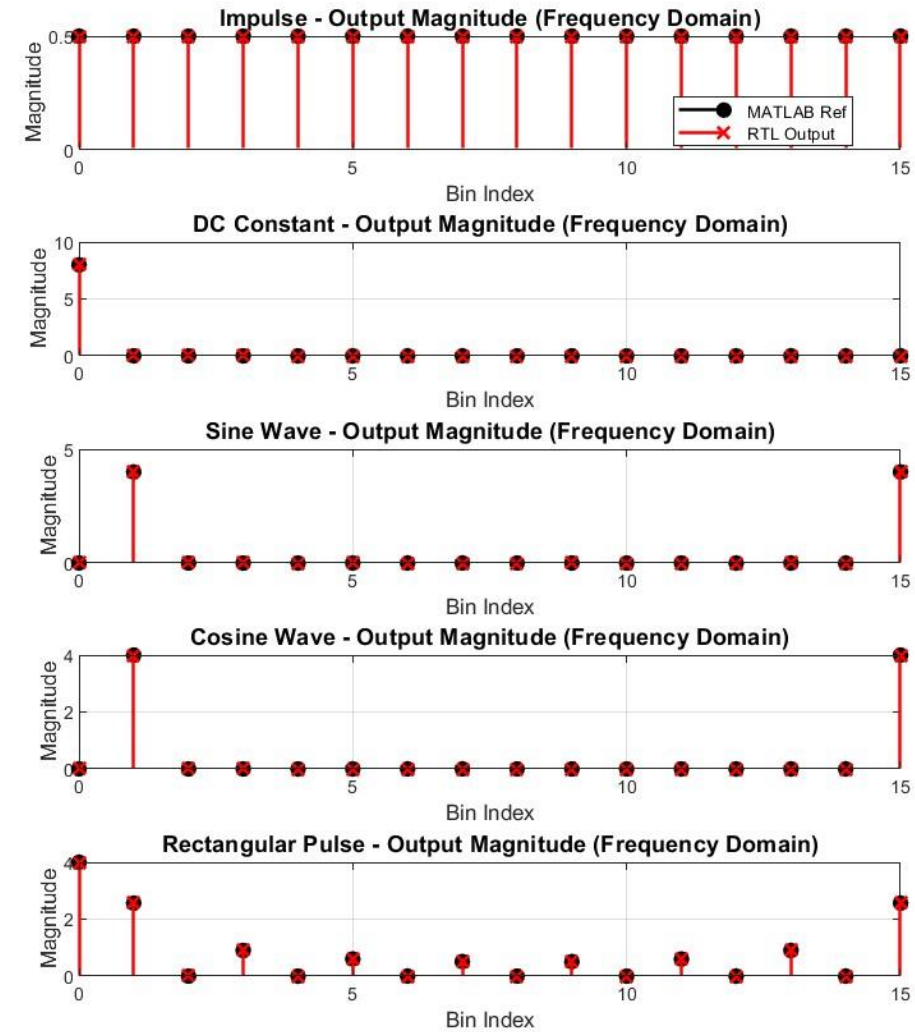
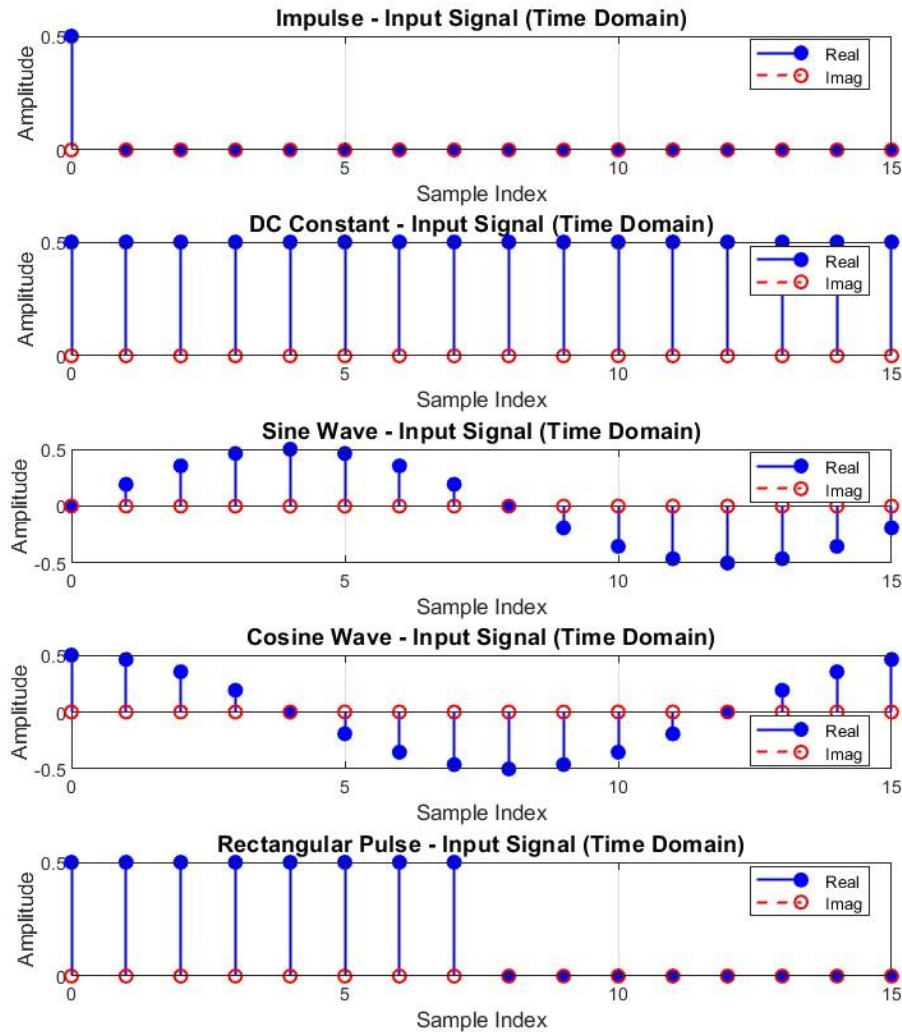
Measured Results:

- Latency value = time of first output - time of first input = 160 ns (15+1 clock cycles)

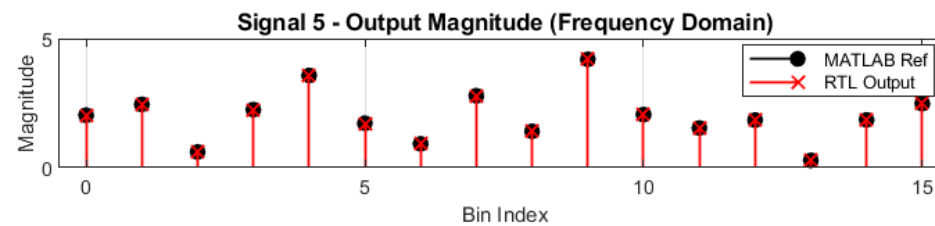
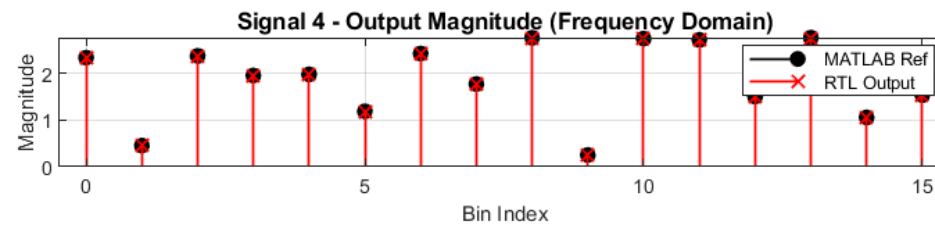
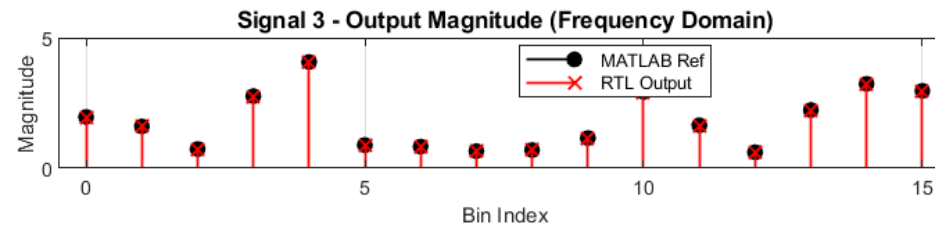
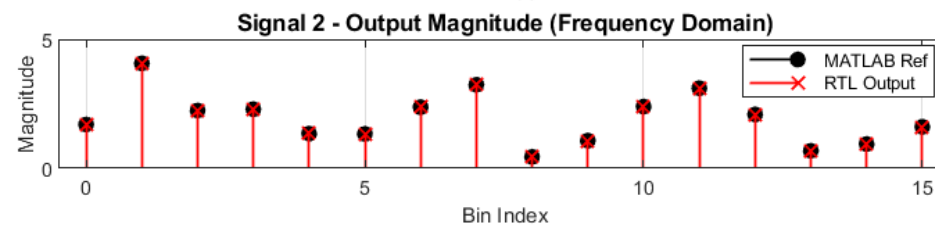
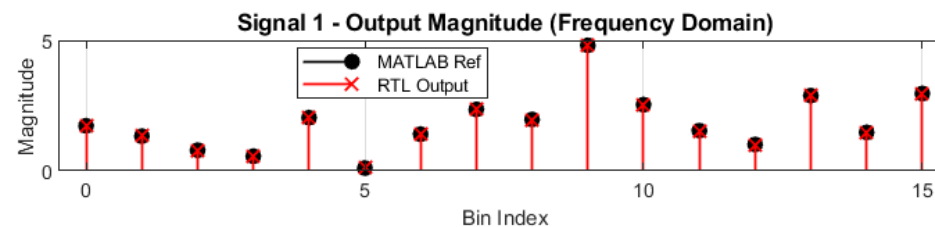
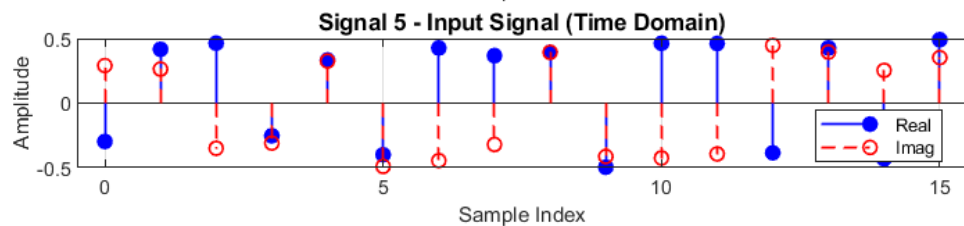
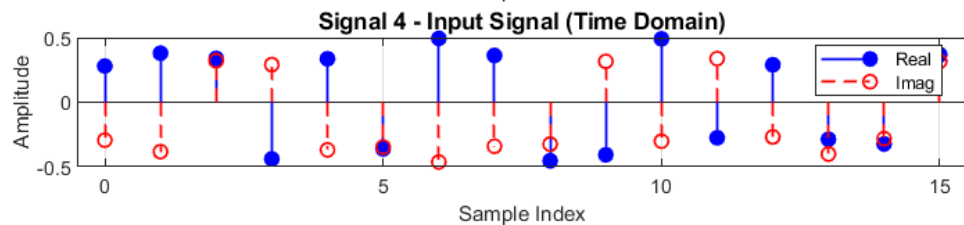
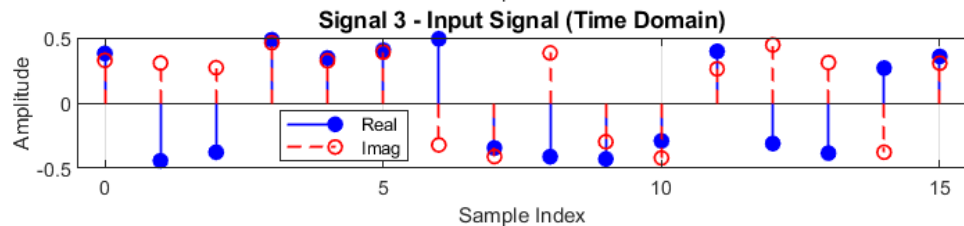
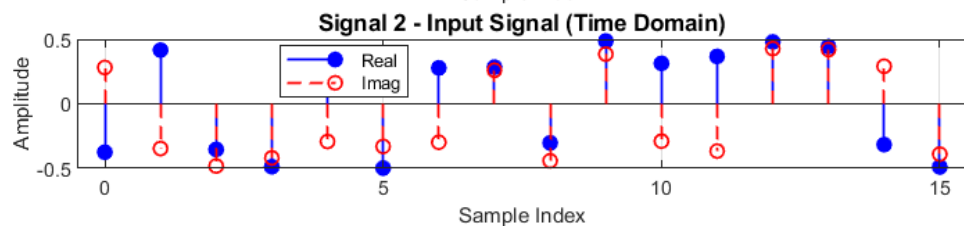
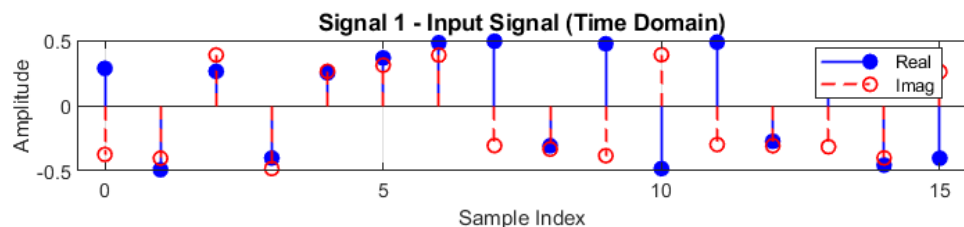


Stimulus: SystemVerilog randomization + input forcing. **Checking:** File-based I/O capture, analyzed in MATLAB for error and SQNR.

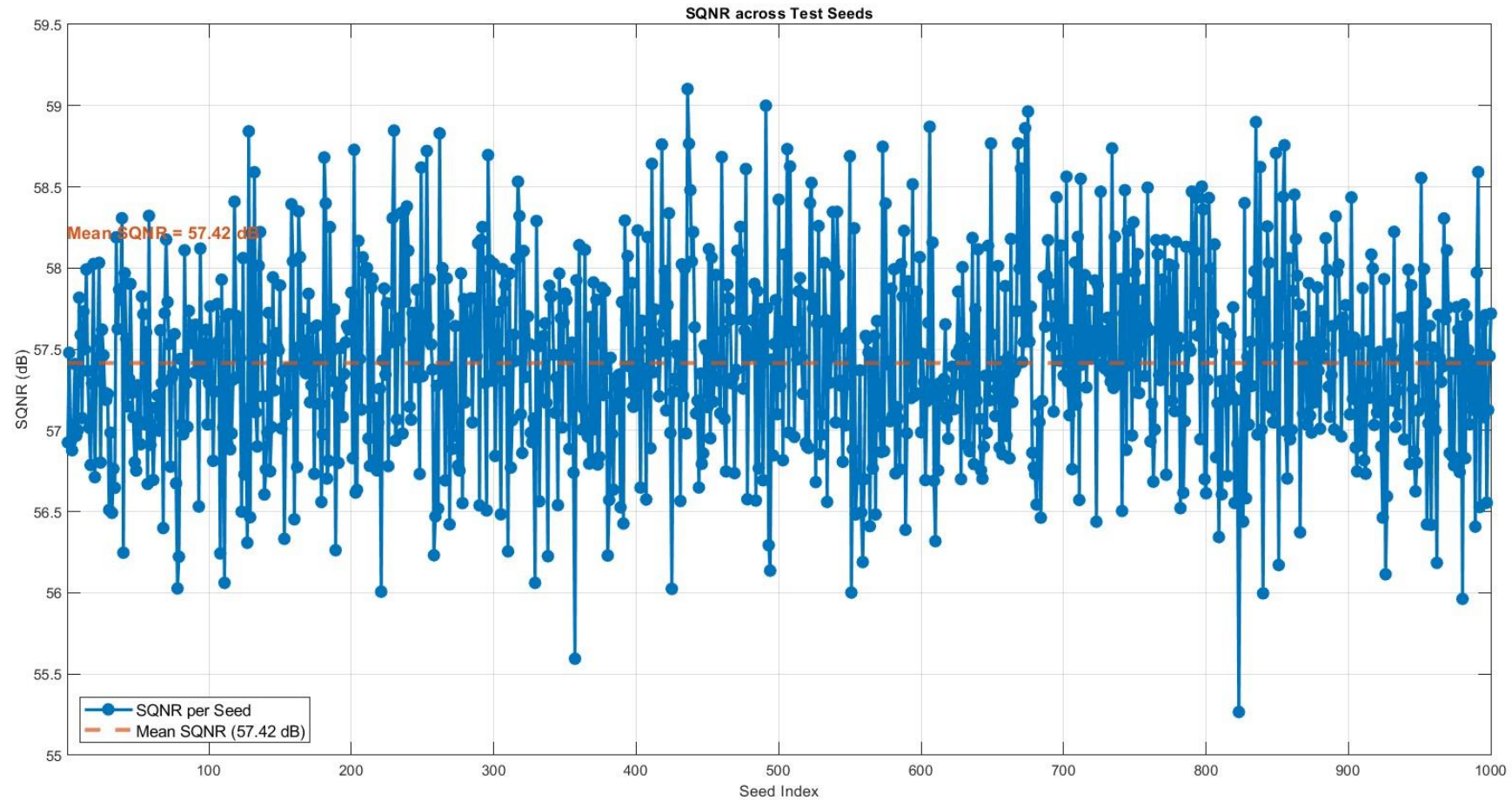
Verification: Five Known Signals



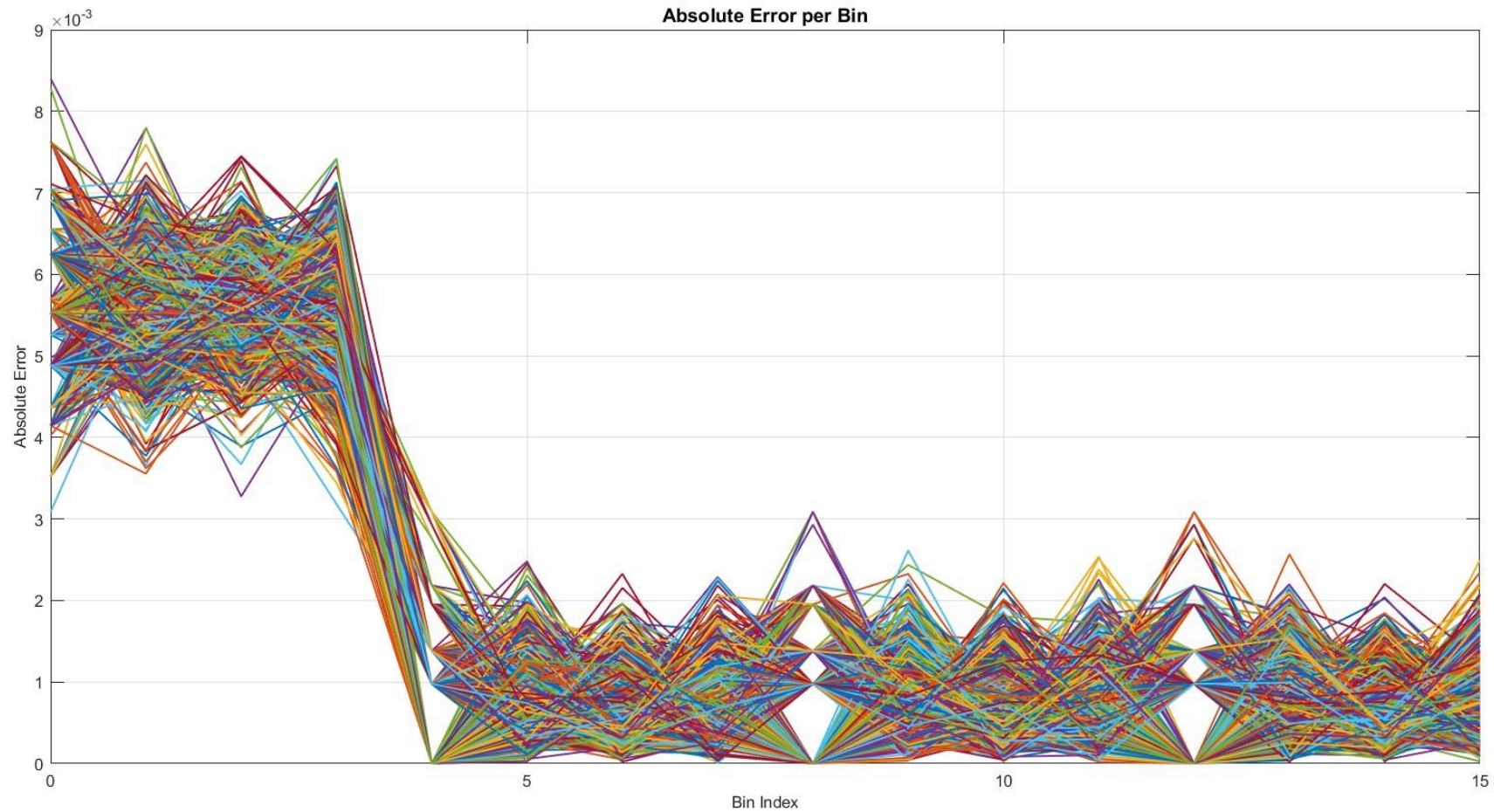
Verification: Five Random Signals



Verification: Random Signals - SQNR



Verification: Random Signals - Error



CoCotb Verification

```
Transaction 500/506 [ RANDOM ] : PASS | SQNR=55.51 dB | MaxErr=0.005591
Transaction 501/506 [ RANDOM ] : PASS | SQNR=55.64 dB | MaxErr=0.006824
Transaction 502/506 [ RANDOM ] : PASS | SQNR=55.73 dB | MaxErr=0.005386
Transaction 503/506 [ RANDOM ] : PASS | SQNR=54.81 dB | MaxErr=0.005626
Transaction 504/506 [ RANDOM ] : PASS | SQNR=54.18 dB | MaxErr=0.006253
Transaction 505/506 [ RANDOM ] : PASS | SQNR=55.06 dB | MaxErr=0.006253
Transaction 506/506 [ RANDOM ] : PASS | SQNR=54.76 dB | MaxErr=0.006338
```

FUNCTIONAL COVERAGE REPORT

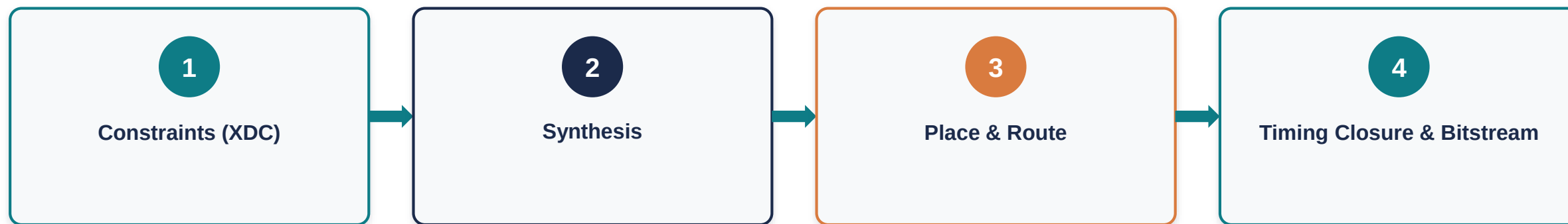
```
Input RE value bins : {'neg_max': 2036, 'neg_mid': 2029, 'zero': 25, 'pos_mid': 1971, 'pos_max': 2035}
Input IM value bins : {'neg_max': 2050, 'neg_mid': 1984, 'zero': 31, 'pos_mid': 1949, 'pos_max': 2082}
Corner cases hit    : {'re_min': True, 're_max': True, 're_zero': True, 'im_min': True, 'im_max': True, 'im_zero': True}
SQNR distribution   : {'<20dB': 0, '20-30dB': 0, '30-40dB': 0, '>40dB': 506}
RE bin coverage     : 5/5 (100.0%)
IM bin coverage     : 5/5 (100.0%)
Corner coverage     : 6/6 (100.0%)
Overall coverage    : 16/16 (100.0%)
```

FINAL REPORT

```
Total transactions : 506 (6 directed + 500 random)
Passed              : 506
Failed              : 0
Mean SQNR           : 55.20 dB
Worst-case SQNR     : 52.71 dB
Max Abs Error       : 0.011049
Functional Coverage : 100.0%
Overall Result      : PASS
```

Taking the Verified FFT Design From RTL to Hardware on ZedBoard

With the FFT design functionally verified through SystemVerilog-based simulation, the next stage moves it toward real hardware. This section covers synthesizing and implementing the design for a Zynq-7020 device, closing timing at the target clock frequency, and generating the final bitstream targeting a ZedBoard.



FPGA Implementation: Constraints (XDC)

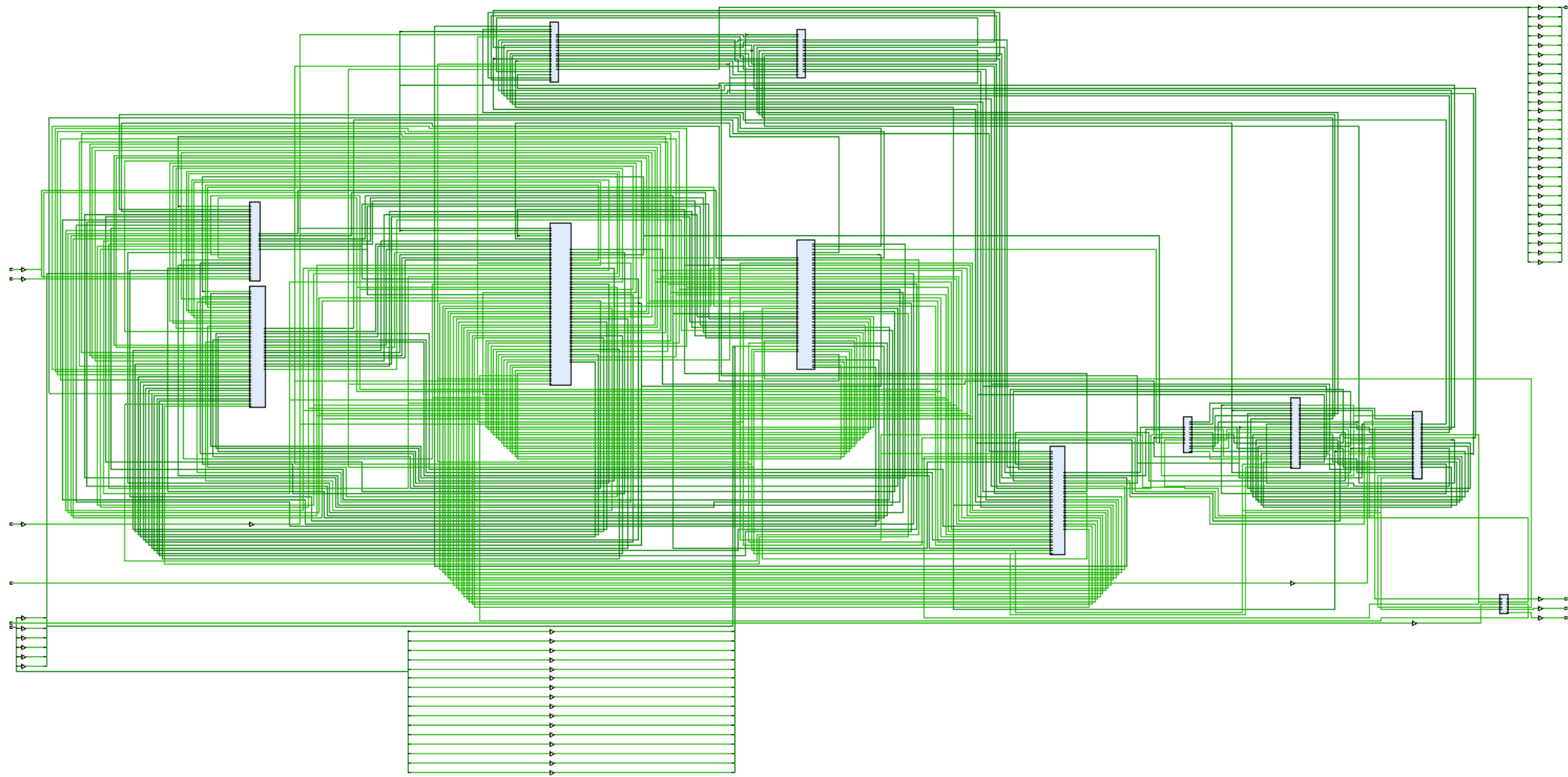
```
create_clock -name clk -period 25 [get_ports clk]

set_false_path -from [get_ports {rst_n}]
set_input_delay -clock [get_clocks clk] -max 2.000 [get_ports {rst_n}]
set_input_delay -clock [get_clocks clk] -min 1.000 [get_ports {rst_n}]

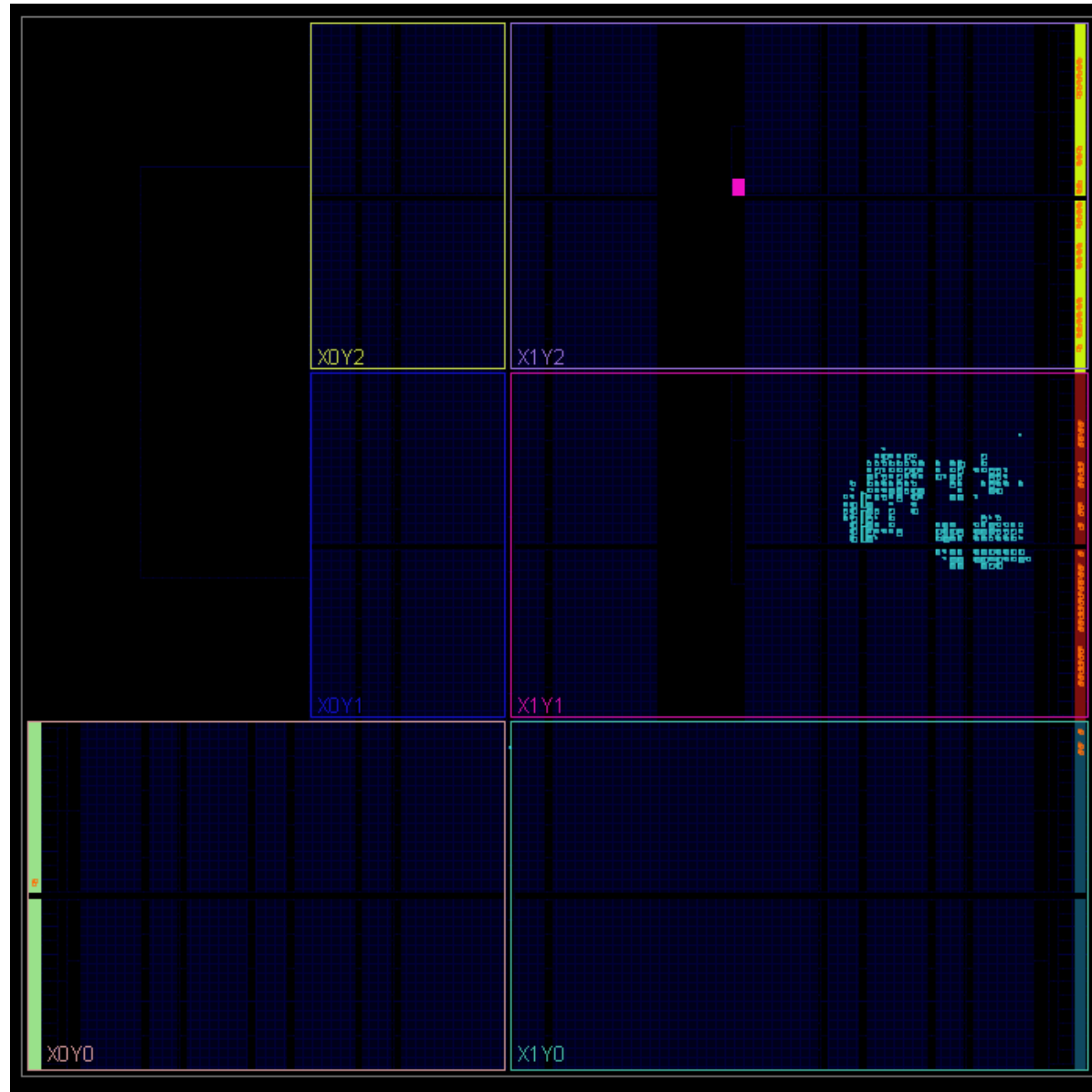
set_input_delay -clock [get_clocks clk] -max 2.000 [get_ports {x_in[*]}]
set_input_delay -clock [get_clocks clk] -min 1.000 [get_ports {x_in[*]}]
set_input_delay -clock [get_clocks clk] -max 2.000 [get_ports {x_start}]
set_input_delay -clock [get_clocks clk] -min 1.000 [get_ports {x_start}]
set_input_delay -clock [get_clocks clk] -max 2.000 [get_ports {x_valid}]
set_input_delay -clock [get_clocks clk] -min 1.000 [get_ports {x_valid}]
set_input_delay -clock [get_clocks clk] -max 2.000 [get_ports {x_done}]
set_input_delay -clock [get_clocks clk] -min 1.000 [get_ports {x_done}]

set_output_delay -clock [get_clocks clk] -max 2.000 [get_ports {y_out[*]}]
set_output_delay -clock [get_clocks clk] -min 1.000 [get_ports {y_out[*]}]
set_output_delay -clock [get_clocks clk] -max 2.000 [get_ports {y_start}]
set_output_delay -clock [get_clocks clk] -min 1.000 [get_ports {y_start}]
set_output_delay -clock [get_clocks clk] -max 2.000 [get_ports {y_valid}]
set_output_delay -clock [get_clocks clk] -min 1.000 [get_ports {y_valid}]
set_output_delay -clock [get_clocks clk] -max 2.000 [get_ports {y_done}]
set_output_delay -clock [get_clocks clk] -min 1.000 [get_ports {y_done}]
```

FPGA Implementation: Synthesis



FPGA Implementation: Place & Route

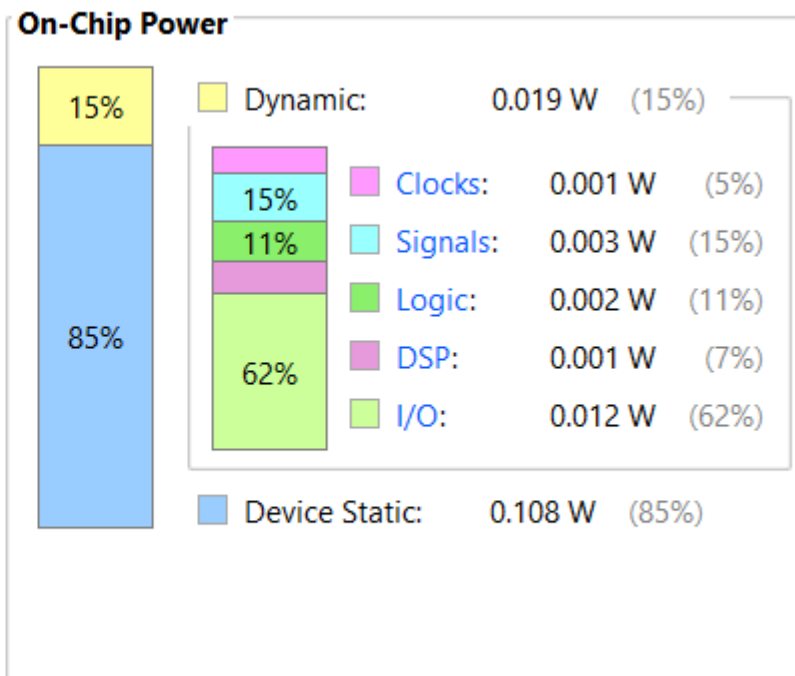


FPGA Implementation: Final reports

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.127 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 26.5°C
Thermal Margin: 58.5°C (4.9 W)
Effective θ_{JA} : 11.5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.004 ns	Worst Hold Slack (WHS): 0.124 ns	Worst Pulse Width Slack (WPWS): 11.520 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 807	Total Number of Endpoints: 807	Total Number of Endpoints: 285

All user specified timing constraints are met.

FPGA Implementation: Final reports

Report Instance Areas:

+-----+-----+-----+-----+			
	Instance	Module	Cells
+-----+-----+-----+-----+			
1	top		952
2	u_bf1	BF_1	20
3	u_bf2	BF_2	16
4	u_bf3	BF_1__parameterized0	67
5	u_bf4	BF_2__parameterized0	16
6	u_ctrl	control_unit	41
7	u_mult	cplx_multiplier	32
8	u_sr1	shift_reg__parameterized2	96
9	u_sr2	shift_reg__parameterized1	130
10	u_sr4	shift_reg__parameterized0	190
11	u_sr8	shift_reg	209
12	u_sync_counter	sync_counter	76
+-----+-----+-----+-----+			

Report Cell Usage:

+-----+-----+-----+		
	Cell	Count
+-----+-----+-----+		
1	BUFG	1
2	CARRY4	76
3	DSP48E1	2
4	DSP48E1_1	2
5	LUT1	28
6	LUT2	211
7	LUT3	7
8	LUT4	6
9	LUT5	202
10	LUT6	83
11	SRL16E	53
12	FDCE	170
13	FDRE	53
14	IBUF	27
15	OBUF	31
+-----+-----+-----+		

40 MHz

Clock Frequency

26

Input Pins

31

Output Pins

FMC

I/O Interface

Pin Breakdown

Inputs (26):

- 22-bit x_in data bus, x_start, x_valid, x_done, rst_n

Outputs (31):

- 28-bit y_out data bus, y_start, y_valid, y_done

Platform Notes

- Data buses (x_in / y_out) routed through the FMC Expansion Connector
- Reset (rst_n) mapped to onboard DIP switch SW0
- Status signals (start / valid / done) mapped to onboard LEDs (LD0–LD2)

► Environment & Technology

Framework: OpenLane Automated RTL-to-GDSII Flow

Technology Node: SkyWater 130nm Open-Source PDK (Sky130)

► Key Toolchain: Yosys, OpenROAD, KLayout

- While this design successfully closed timing on a Zynq-7020 FPGA targeting a ZedBoard, a direct 1:1 performance comparison with this ASIC implementation is technically invalid. The ASIC flow utilizes a legacy 130nm educational node (Sky130), which naturally lacks the performance characteristics of the optimized, more modern silicon found in the Zynq-7020. Consequently, rather than benchmarking raw speed and area against the FPGA, this phase of the project strictly prioritizes mastering the physical design methodology and the RTL-to-GDSII flow.

ASIC Implementation: Setup

Config.json

```
{
  "DESIGN_NAME": "FFT_2_2_top",
  "VERILOG_FILES": "dir::src/*.v",

  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 40,

  "BASE_SDC_FILE": "dir::src/FFT_2_2_top_fpga_style .sdc",

  "RUN_HEURISTIC_DIODE_INSERTION": 1,
  "GRT_ANT_ITERS": 5,
  "GRT_ANT_MARGIN": 10,
  "DIODE_ON_PORTS": "in",

  "FP_IO_MIN_DISTANCE": 4,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_CORE_UTIL": 60,
  "FP_PDN_VPITCH": 30,
  "FP_PDN_HPITCH": 30,

  "FP_CORE_UTIL": 40,
  "PL_TARGET_DENSITY_PCT": "expr::($FP_CORE_UTIL + 10.0)",

  "MAX_FANOUT_CONSTRAINT": 15,
  "MAX_TRANSITION_CONSTRAINT": 0.5,
  "OUTPUT_CAPACITANCE_LOAD": 1.5
}
```

Pin Order

```
#N
clk
rst_n
x_valid
x_start
x_done

#S
y_valid
y_start
y_done

#W
x_in.*

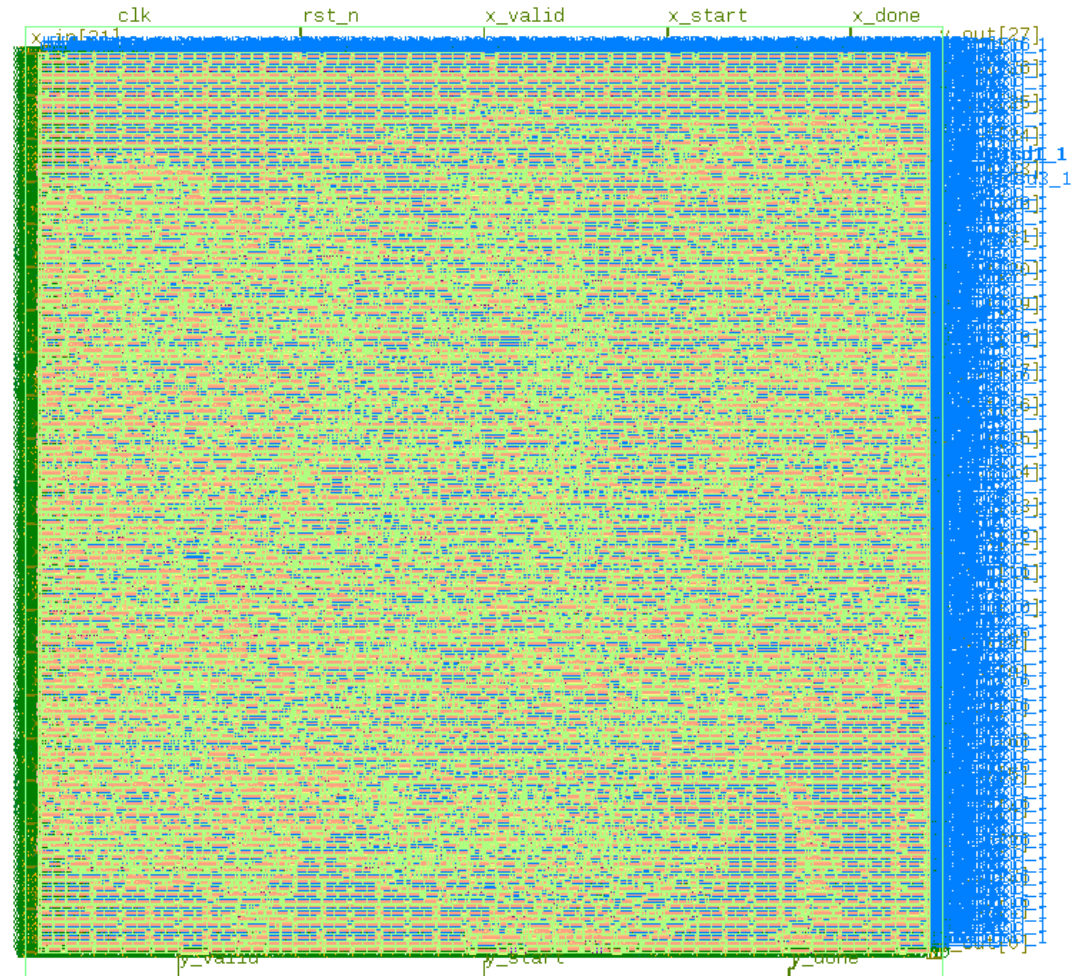
#E
y_out.*
```

ASIC Implementation: Synthesis

Number of wires:	6151	sky130_fd_sc_hd__dfrtp_2	517
Number of wire bits:	6199	sky130_fd_sc_hd__inv_2	113
Number of public wires:	524	sky130_fd_sc_hd__mux2_2	490
Number of public wire bits:	572	sky130_fd_sc_hd__nand2_2	492
Number of memories:	0	sky130_fd_sc_hd__nand3_2	59
Number of memory bits:	0	sky130_fd_sc_hd__nand3b_2	3
Number of processes:	0	sky130_fd_sc_hd__nand4_2	10
Number of cells:	6172	sky130_fd_sc_hd__nor2_2	417
sky130_fd_sc_hd__a2111o_2	5	sky130_fd_sc_hd__nor3_2	43
sky130_fd_sc_hd__a2111oi_2	2	sky130_fd_sc_hd__nor3b_2	3
sky130_fd_sc_hd__a211o_2	38	sky130_fd_sc_hd__nor4_2	2
sky130_fd_sc_hd__a211oi_2	30	sky130_fd_sc_hd__o2111a_2	1
sky130_fd_sc_hd__a21bo_2	70	sky130_fd_sc_hd__o211a_2	64
sky130_fd_sc_hd__a21boi_2	26	sky130_fd_sc_hd__o211ai_2	18
sky130_fd_sc_hd__a21o_2	251	sky130_fd_sc_hd__o21a_2	182
sky130_fd_sc_hd__a21oi_2	208	sky130_fd_sc_hd__o21ai_2	190
sky130_fd_sc_hd__a221o_2	3	sky130_fd_sc_hd__o21ba_2	31
sky130_fd_sc_hd__a22o_2	80	sky130_fd_sc_hd__o21bai_2	19
sky130_fd_sc_hd__a22oi_2	4	sky130_fd_sc_hd__o221a_2	9
sky130_fd_sc_hd__a2bb2o_2	51	sky130_fd_sc_hd__o22a_2	26
sky130_fd_sc_hd__a311o_2	6	sky130_fd_sc_hd__o22ai_2	3
sky130_fd_sc_hd__a311oi_2	2	sky130_fd_sc_hd__o2bb2a_2	13
sky130_fd_sc_hd__a31o_2	85	sky130_fd_sc_hd__o311a_2	5
sky130_fd_sc_hd__a31oi_2	23	sky130_fd_sc_hd__o31a_2	23
sky130_fd_sc_hd__a32o_2	7	sky130_fd_sc_hd__o31ai_2	28
sky130_fd_sc_hd__a41oi_2	1	sky130_fd_sc_hd__o32a_2	6
sky130_fd_sc_hd__and2_2	213	sky130_fd_sc_hd__o32ai_2	2
sky130_fd_sc_hd__and2b_2	162	sky130_fd_sc_hd__o41a_2	1
sky130_fd_sc_hd__and3_2	108	sky130_fd_sc_hd__o41ai_2	1
sky130_fd_sc_hd__and3b_2	9	sky130_fd_sc_hd__or2_2	279
sky130_fd_sc_hd__and4_2	7	sky130_fd_sc_hd__or2b_2	127
sky130_fd_sc_hd__and4b_2	1	sky130_fd_sc_hd__or3_2	73
sky130_fd_sc_hd__and4bb_2	3	sky130_fd_sc_hd__or3b_2	19
sky130_fd_sc_hd__buf_1	670	sky130_fd_sc_hd__or4_2	13
		sky130_fd_sc_hd__or4b_2	6
		sky130_fd_sc_hd__or4bb_2	5
		sky130_fd_sc_hd__xnor2_2	603
		sky130_fd_sc_hd__xor2_2	211

Chip area for module '\FFT_2_2_top': 64574.432000

ASIC Implementation: GDSII



ASIC Implementation: Final reports

```
=====
report_power
=====
===== Typical Corner =====
=====
```

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	9.09e-04	1.89e-04	5.92e-09	1.10e-03	1.0%
Combinational	1.18e-02	9.22e-02	3.73e-08	1.04e-01	99.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%

Total	1.27e-02	9.24e-02	4.32e-08	1.05e-01	100.0%
	12.1%	87.9%	0.0%		

```
=====
report_tns
=====
tns 0.00

=====
report_wns
=====
wns 0.00

=====
report_worst_slack -max (Setup)
=====
worst slack 4.65

=====
report_worst_slack -min (Hold)
=====
worst slack 0.19
```

ASIC Implementation: Summary

✓ Flow Completed

✓ GDSII Generated

✓ 0 DRC Errors

✓ 0 LVS Errors

Physical Implementation

Die area	0.177 mm ²
Core area	162,586 μm ²
Core utilization (before routing)	41.2 %
Standard cells (synthesized)	6,385
Total cells (incl. fill/tap/decap)	24,445
Routed wire length	212,182 μm
Vias	50,342

Signoff Checks

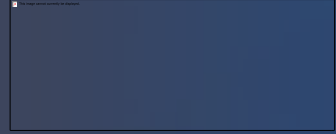
DRC violations (Magic + KLayout)	0
LVS errors	0
Antenna violations (pin/net)	0 / 0
Routing shorts / spacing	0 / 0
Total power (typical corner)	≈ 0.105 μW
— internal / switching	0.0127 / 0.0924 μW

Timing Summary

Target clock period	40.0 ns (25 MHz)
Critical path delay	33.43 ns
SPEF Worst negative slack (WNS)	0 ns
SPEF Total negative slack (TNS)	0 ns

- ▶ **Scale FFT Architecture:** Upgrade the design to support higher transform sizes, such as 64-point and 256-point FFTs.
- ▶ **UVM verification**

- ▶ S. He and M. Torkelson, "A new approach to pipeline FFT processor," in Proceedings of IPPS '96: 10th International Parallel Processing Symposium, Honolulu, HI, USA, 1996, pp. 766-770.
- ▶ Y. S. Algnabi, A. Aldaamee, R. Teymourzadeh, M. Othman, and M. M. Islam, "Novel architecture of pipeline Radix-2² SDF FFT based on digit-slicing technique," Journal of Signal Processing Systems (or specify Conference/Journal Name if applicable).
- ▶ U. Meyer-Baese, Digital Signal Processing with Field Programmable Gate Arrays, 4th ed., Berlin, Germany: Springer-Verlag, 2014.



Thank You